Lab 1:

Introduction of 8086 Microprocessor Architecture and Emulator

Emu8086 combines an advanced source editor, assembler, disassembler, software emulator (Virtual PC) with debugger.

Visual interface is very easy to work with. You can watch registers, flags and memory while your program executes. Arithmetic & Logical Unit (ALU) shows the internal work of the central processor unit (CPU).

Emulator runs programs on a Virtual PC, this completely blocks your program from accessing real hardware, such as hard-drives and memory, since your assembly code runs on a virtual machine, this makes debugging much easier.

EMU8086 - Microprocessor Emulator is extremely helpful for those who just begin to study assembly language. It compiles the source code and executes it on emulator step by step.

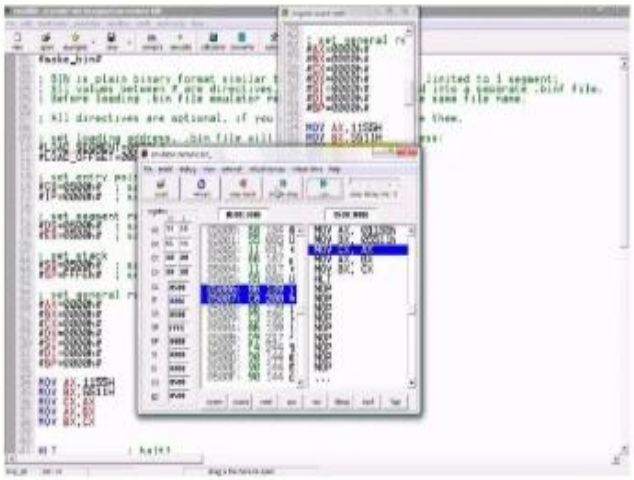


Figure: Interface of emu8086

Where to start?

1. Click **code examples** and select **Hello, world**. A code example with many comments should open. All comments are green and they take up about **90%** of all text, so don't be scared by this tiny *"Hello Word"* code. The compiled executable is only about **100** bytes long, because it uses no interrupts and has only one loop for color *highlighting* the text. All other code is straight-forward and writes directly to video memory.

2. To run this example in the emulator, click **emulate** (or press **F5**). The program then attmepts to assemble and save the executable to **c:\emu8086\MyBuild**. If the assembler succeeds in creating the file, the emulator will also automatically load it into memory.

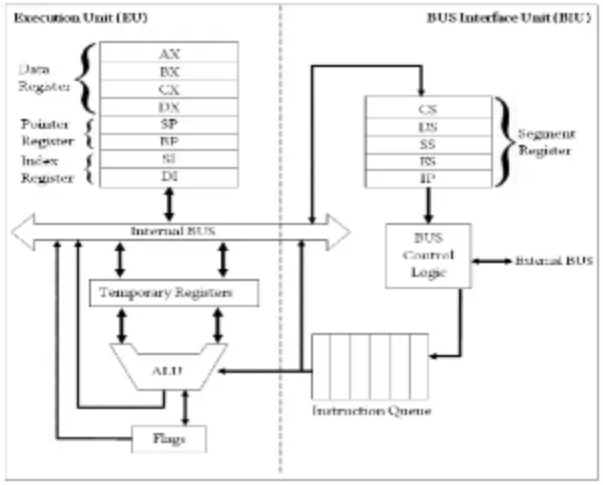
3. You can then click **single step** (or press **F8**) to step through the code one instruction at a time, observing changes in registers and the emulator screen. You can also click **step back** (or press **F6**) to see what happens when reversing those changes.

4. There are many ways to print *"Hello, World"* in assembly language, and this certainly isn't the shortest way. If you click **examples** and browse **c:\emu8086\examples**, you'll find **HelloWorld.asm** which assembles into only a **30**-byte executable. Unlike the previous example which carries out each step by itself, this one is much smaller because it uses a built-in interrupt function of the operating system to write to the display.

The integrated 8086 assembler can generate console programs that can be executed on any computer that runs x86 machine code (Intel/AMD architecture)

The architecture of the 8086 Intel microprocessor is called "Von Neumann architecture" after the mathematician who conceived of the design.

NOTE: A CPU can interpret the contents of memory as either instructions or data; there's no difference in the individual bytes of memory, only the way in which they're arranged. Because of this, it's even possible for programs to re-write their own instructions, then execute the instructions they've changed.



Execution unit receives program instruction codes and data from the BIU, executes them and stores the results in the general registers. It can also store the data in a memory location or send them to an I/O device by passing the data back to the BIU. This unit, EU, has no connection with the system Buses. It receives and outputs all its data through BIU.

`ALU (Arithmetic and Logic Unit)` : The EU unit contains a circuit board called the Arithmetic and Logic Unit. The ALU can perform arithmetic, such as, +,-,×,/ and logic such as OR, AND, NOT operations.
`Registers` : A register is like a memory location where the exception is that these are denoted by name rather than numbers. It has 4 data registers, AX, BX, CX, DX and 2 pointer registers SP, BP and 2 index registers SI, DI and 1 temporary register and 1 status register FLAGS .

AX, BX, CX and DX registers has 2 8-bit registers to access the high and low byte data registers. The high byte of AX is called AH and the low byte is AL. Similarly, the high and low bytes of BX, CX, DX are BH and BL, CH and Cl, DH and DL respectively. All the data, pointer, index and status registers are of 16 bits. Else these, the temporary register holds the operands for the ALU and the individual bits of the FLAGS register reflect the result of a computation.

Bus Interface Unit:

As the EU has no connection with the system Busses, this job is done by BIU. BIU and EU are connected with an internal bus. BIU connects EU with the memory or I/O circuits. It is responsible for transmitting data, addresses and control signal on the busses.
`Registers` : BIU has 4 segment busses, CS, DS, SS, ES. These all 4 segment registers holds the addresses of instructions and data in memory. These values are used by the processor to access memory locations. It also contain 1 pointer register IP. IP contains the address of the next instruction to executed by the EU.
`Instruction Queue` : BIU also contain an instruction queue. When the EU executes instructions, the BIU gets up to 6 bytes of the next instruction and stores them in the instruction queue and this process is called instruction prefetch. This is a process to speed up the processor. Also when the EU needs to be connected with memory or peripherals, BIU suspends instruction prefetch and performs the needed operations.

** Simple program to show "Hello world" message

```
.MODEL SMALL
.STACK 100H
.DATA
STR1 DB 'Hello World!$'

.CODE
MAIN PROC
   MOV AX,@DATA
   MOV DS,AX
```

```
    LEA DX,STR1
    MOV AH,9
    INT 21H


    MOV AH,4CH
    INT 21H
    MAIN ENDP
END MAIN


;Title:- Input Two Number & Print them In Assembly Language

;Purpose: Solve Some Easy Problem in Assembly Language


PRINT MACRO P
    MOV AH,9
    LEA DX,P            ;PRINT str
    INT 21H
    ENDM

.MODEL SMALL
.STACK 100H

.DATA
str1 DB "Please Enter First Number: $"
str2 DB "Please Enter Second Number: $"
str3 DB "Your Given First Number is: $"
str4 DB "Your Given Second Number is: $"


.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    PRINT str1
    MOV AH,1
    INT 21H
    MOV BL,AL

    CALL New_Line

    PRINT str2
    MOV AH,1
    INT 21H
    MOV CL,AL

    CALL New_Line

    PRINT str3
```

```
    MOV AH,2
    MOV DL,BL
    INT 21H

    CALL New_Line

    PRINT str4
    MOV AH,2
    MOV DL,CL
    INT 21H

MOV AH,4CH
INT 21H

ENDP


New_Line PROC
    MOV AH,2
    MOV DX,10
    INT 21H
    MOV AH,2            ;CALL New_Line
    MOV DX,13
    INT 21H
    RET
    New_Line ENDP


END MAIN
```